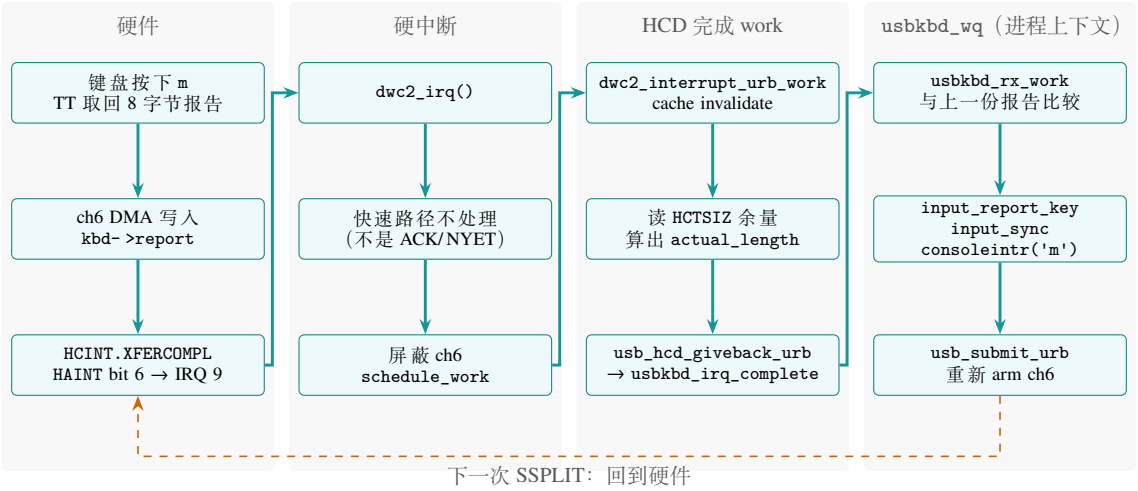


图 7.13: 一份键盘报告从硬件到 shell 的路径。每一列是一种执行上下文：硬中断只做与微帧时序有关的最少工作；HCD 的完成 work 负责缓存维护并把 URB 归还类驱动；HID 解析、输入事件和控制台都在键盘自己的 workqueue 里完成，最后重新提交 URB。键盘回 NAK（没有新按键）时，HCD completion 只登记 `bInterval` 期限，到期后由专用 DWC2 periodic workqueue 重发，URB 不归还类驱动

图 7.14 把同一条路径按时间顺序摊开，并分出两种结果：键盘回 NAK 时 URB 始终不归还类驱动，只有拿到完整报告才走 `usb_hcd_giveback_urb()`。

NAK 的无忙等周期重试. ch6 的最终 IRQ 仍通过 `schedule_work(&interrupt_rx.work)` 立即进入公共 `system_wq`。`dwc2_interrupt_urb_work()` 调用 `dwc2_interrupt_rx_complete()` 读取并清除 `HCINT`；若本轮是 NAK，或 CSPLIT 已越过当前 frame 的有效窗口，它不再执行毫秒级 `udelay(bInterval)`，而是调用：

```
mod_delayed_work(&dwc2_periodic_wq, &interrupt_rx.retry_work,
                 interval_jiffies);
```

这时 `retry_work` 先进入 workqueue 核心按 `expires` 排序的全局 `delayed_head`，并未进入可执行队列。CPU0 的 10 ms timer IRQ 调用 `workqueue_timer_tick()` 推进 `work_jiffies`；到期后才把其中的普通 `work_struct` 移入只有一个 kworker 的 `dwc2_periodic_wq`。该 worker 执行 `dwc2_interrupt_retry_work()`，从新的 SSPLIT 重新 arm ch6。等待期间没有 kworker 运行，原 URB 始终保持 submitted，也不会调用键盘类驱动。

拔出/取消时先在 `usb_lock` 下清除 `interrupt_rx.active` 并停止 ch6，再在锁外同步取消 delayed work：

```
cancel_delayed_work_sync(&interrupt_rx.retry_work);
```

到期回调也要取得 `usb_lock`，因此同步取消绝不能在持锁状态下等待；这个顺序同时保证取消返回后不会有旧 retry 重新 arm 已经释放的 USB 端点。

拔出设备时顺序相反：`usbkbd_remove()` 先把 `active=0`、`disconnected=1`，再 `usb_kill_urb()`——它停掉 ch6 的 DMA、屏蔽 `HCINTMSK/HAINTMSK` 对应位、同步取消 HCD 自己的完成 work，并等待正在运行的 completion 返回；然后 `cancel_work_sync(&kbd->rx_work)`；最后才注销 input 设备、释放 URB 和这份私有状态。这两步的返回语义就是保证本身：`usb_kill_urb()` 返回后不会再有 DWC2 completion 摸到这只 URB，`cancel_work_sync()` 返回后再也没有键盘下半部访问设备私有内存，正对应 Linux 在 `disconnect()` 里对 `usb_kill_urb()` 的要求。

7.4.7 HID 报告与输入子系统

一份 8 字节报告到手，`usbkbd_rx_work()` 要同时喂两个消费者：evdev 要的是“哪个键按下、哪个松开”，控制台要的是“打出了什么字符”。两者的判断都建立在同一个动作上——把新报告和 `kbd->previous` 逐槽比较。唯一需要提前特判的是 rollover：按键多到键盘自己无法分辨时，六个键槽会被全部填成错误 usage（1 ~ 3），